

Malas prácticas en programación

Las siguientes son prácticas que se consideran inadecuadas en esta asignatura de programación, en cuanto a que dificultan algún criterio de calidad de los considerados en ella. Se pueden escribir programas correctos utilizándolas, pero incidirán negativamente en la claridad de código, documentación y mantenibilidad.

No es una lista exhaustiva, y se refiere solamente a la parte de la asignatura vista hasta el momento.

1. Nombrado inadecuado

- Los nombres de variables, constantes, métodos ... no son adecuados en cuanto a
 - significación: deberían ser significativos. El nombre sugiere el significado.
 - consistencia: se debería seguir la misma estrategia en todo el código (“camel case”, con subrayados ...)
 - convenciones: se deberían mantener las habituales (constantes en mayúsculas, clases comienzan con mayúscula, variables y métodos con minúscula ...)
 - longitud (no deberían ser demasiado largos)
- Afecta a la claridad de código y mantenibilidad, y en ciertos casos es más propenso a errores.
- Excepciones: índices de control de bucle (típicamente $i, j \dots$), fórmulas conocidas, cálculos “ad hoc”.

2. Código inactivo “*dead code*”

- Un fragmento de código que no se ejecuta nunca. Puede ser un resto de versiones anteriores o sencillamente un error.
- En algunos casos los compiladores lo detectan y advierten.

3. Constantes mágicas

- Consiste en usar valores constantes directamente en el código fuente.
- Suele proceder de un defecto en la fase de análisis (detección de constantes del problema) o de diseño.
- Aunque suele hacer el código menos verboso, más corto, y en primera aproximación más legible, tiene como desventajas:
 - Puede no estar claro el significado de la constante
 - Incrementa la posibilidad de error tipográfico en caso de duplicación

- Dificulta la modificabilidad si hay duplicación : habrá que modificar la constante en varios lugares distinguiendo en qué casos porcede hacerlo, dado que puede aparecer la misma constante en el código con distintos significados
 - Puede influir en otras constantes silenciosamente
 - Dificulta la parametrización
- La ejecución es aparentemente más eficiente, pero en general los compiladores resuelven ese problema
 - La solución es darle un nombre (significativo) a la constante y utilizar el nombre en donde proceda. La definición puede ser global (para todos los métodos de la clase), definida en la clase, antes de todos los métodos, incluso del “main”.
 - Excepciones admisibles
 - 0 y 1 como valores de iniciación o incremento de bucles “for”
 - 2 para detectar paridad
 - fórmulas conocidas ($2 * Math.PI * radio$, $b * b - 4 * a * c \dots$)
 - 100 para porcentajes
 - potencias de 10 para conversiones del sistema métrico decimal
 - valores de conversión de unidades de tiempo o de ángulos (60, 24 ...)
 - números o letras de elección de opción
 - mensajes de salida, “Si”, “No” ...
 - Ejemplo : contar experimentalmente cuántas veces sale el valor máximo en tiradas aleatorias de un dado de 12 caras

```
int veces = 0, dado;
for (int i = 1; i<= 12000; i++) {
    dado = (int)(Math.random()*12);
    if ( dado == 11 )
        veces ++;
}
System.out.println (
    11 + " ha salido " +
    veces + " veces");
```

```
final int TIRADAS = 1200;
final int NCARAS = 12;
final int OBJETIVO = NCARAS -1;

int veces = 0, dado;
for (int i = 1; i<= TIRADAS; i++) {
    dado = (int)(Math.random()*NCARAS);
    if ( dado == OBJETIVO )
        veces ++;
}
System.out.println (
    OBJETIVO + " ha salido " +
    veces + " veces");
```

4. Mal uso de comentarios

- Que no los haya, que sean pocos, que sean demasiados, que sean inadecuados o redundantes.
- Consituyen una parte importante de la documentación interna y influyen muy decisivamente en la dificultad de lectura del código.

5. Código largo o desorganizado

- El código desorganizado, con muchas líneas o muy largas, es muy difícil de leer.

- Se recomiendan códigos que no sobrepasen una página y líneas que no sobrepasen los 80 caracteres.
- Un diseño modular facilitará la construcción de métodos cortos, que se podrán considerar independientemente.
- Suele ser posible separar las sentencias en varias líneas, que con un buen sangrado facilitarán la lectura.
- Un sangrado bien elegido y consistente es fundamental. Los entornos de desarrollo (eclipse por ejemplo) ayudan en esta tarea.

6. Duplicación de código

- Consiste en la presencia de líneas de código que se repiten en varios lugares.
- Es proclive a errores y dificulta la modificabilidad.
- Suele ser síntoma de un diseño pobre, poco o mal modularizado.

7. Mala elección de parámetros

- Se refiere a métodos que tienen demasiados parámetros, o son innecesarios.
- En general los métodos deben tener parámetros. Hay algunos (pocos) casos en los que carecen de ellos.
- La carencia de parámetros a veces es consecuencia de otras malas prácticas, como encadenamiento o uso de variables globales.
- Un excesivo número de parámetros tampoco es recomendable.

8. Uso de variables globales

- Consiste en declarar variables fuera de los métodos, a los que todos ellos tendrán acceso para uso y modificación.
- Es tentador: se escribe menos. En los primeros tiempos de la programación se usaba bastante, a veces por no existir otra alternativa mejor.
- Permite que los métodos no tengan parámetros.
- El código resulta muy difícil de mantener y reutilizar.
- Ejemplo

```

public class MAL {
    static int a, b;
    public static void main
        (String args []) {
        a = 1;
        b = 2;
        imprime ();
    }
    public static void imprime
        ()
    {
        System.out.println
            (a + " " + b);
    }
}

```

```

public class BIEN {

    public static void main
        (String args []) {
        int a = 1;
        int b = 2;
        imprime (a, b);
    }
    public static void imprime
        (int x, int y)
    {
        System.out.println
            (x + " " + y);
    }
}

```

9. Mala elección de estructuras alternativas

- secuencia en lugar de alternativa

- Elegir una secuencia de alternativas cuando lo natural sería una alternativa compleja.
- Ejemplo: sumando valores absolutos:

<pre> if (a > 0) sumaAbsolutos += a; if (a < 0) sumaAbsolutos -= a; </pre>	<pre> if (a > 0) sumaAbsolutos += a; else if (a < 0) sumaAbsolutos -= a; </pre>
--	---

- alternativa en lugar de secuencia

- Elegir una alternativa compleja cuando lo natural sería una secuencia de alternativas.
- Ejemplo

<pre> if (mes == 8) { if (diasem == 7) { laborable = false; vacaciones = true; } else { laborable = true; vacaciones = true; } } else { if (diasem == 7) { laborable = false; vacaciones = false; } else { laborable = true; vacaciones = false; } } </pre>	<pre> if (diaSem == 7) laborable = false; else laborable = true; if (mes == 8) vacaciones = true; else vacaciones = false; </pre>
---	--

10. Salir antinaturalmente de una estructura de control

- Consiste en utilizar una sentencia para abandonar una estructura de control desde su interior.
- Ocurre cuando se utiliza una sentencia “break” para salir de un bucle

- O cuando hay una sentencia “return” que no es la última de la secuencia, normalmente bajo una cláusula “if”
- También ocurre cuando se utiliza una sentencia “return” en el interior de un bucle de un método
- En algunas situaciones es más fácil resolver el problema así, y algunas propuestas, incluso dentro de la “programación estructurada”, lo permiten.
- Con frecuencia procede de un “parche” posterior al diseño, por una condición no considerada en su momento.
- No es necesario si se diseña de forma cuidadosa. Cualquier bucle o método se puede reformular sin utilizar este mecanismo. En caso extremo utilizando un bucle controlado por bandera.
- En esta asignatura está explícitamente prohibido.

11. Mala elección de sentencia de iteración

- Elegir una sentencia iterativa inadecuada.
- Teniendo la estructura secuencial y las alternativas simple y doble, solamente se necesitaría una sentencia iterativa, con condición al principio (“while”) o con condición al final (“do ... while”). Los lenguajes proponen con frecuencia ambas o más, para facilitar la tarea del programador y, además, mejorar la legibilidad del código.
- En particular, hay sentencias iterativas específicas para recorrer estructuras llamadas *iterables*, explícitas o no; por ejemplo “for”. Algunos lenguajes restringen su uso a tales recorridos. Pero en Java, “for” funciona realmente como una abreviatura:

```
for (sentA; expLógica; sentB )
    sentC ;
```

equivale a

```
{
    sentA;
    while (expLógica) {
        sentC ;
        sentB ;
    }
}
```

Esto permite escribir, por ejemplo, los 3 siguientes códigos para obtener del usuario un entero positivo, que “funcionan”, pero son de muy distinta calidad:

```
// código 1
do {
    System.out.println("Escriba un número >0");
    n = in.nextInt();
} while (n < 0);

// código 2
n = -1;
while (n < 0) {
    System.out.println("Escriba un número >0");
    n = in.nextInt();
}

// código 3
for (n = -1; n < 0; n = in.nextInt())
    System.out.println("Escriba un número >0");
```

12. Modificar los elementos de control de un bucle “for”

- La sentencia que se itera modifica en algún caso uno de los extremos del bucle o la variable de control
- Cambia completamente el sentido de la estructura iterativa.
- Las consecuencias son impredecibles. Dependen del lenguaje o incluso de la versión del compilador. Se pueden producir resultados distintos, o no compilar ...
- Se encuentra a veces utilizado para salir de un bucle “for” cuando cierta condición se cumple.
- Ejemplo

```
int fin = ... , ini = ..., paso = ...;
for (int i = ini; i<=fin; i+=paso) {
    // acciones usando i
    i++; fin --; paso --;
}
```

13. Caso especial dentro de una iteración por índice

(“*loop-switch sequence*”)

- Ocurre cuando, dentro de la acción de un bucle controlado por índice (“*for*”), se codifica una alternativa que elige en función de tal índice, tratando de forma especial algún caso.
- Normalmente está encubriendo varias iteraciones separadas. El efecto en la lectura del código es engañoso: afecta a la claridad. Romper el bucle en partes lo hace más fácil de leer y se acerca más a la realidad de la ejecución.
- Es ineficiente, porque realiza comprobaciones y pasadas innecesarias.
- Ejemplo

<pre>for (int i = 1; i<=10; i++) { if (i == 5) System.out.println (i); else System.out.print (i + " "); }</pre>	<pre>for (int i = 1; i<=4; i++) { System.out.print (i + " ") } System.out.println (5); for (int i = 6; i<=10; i++) { System.out.print (i + " ") }</pre>
--	---
- Hay algunas excepciones en las que tiene sentido, por ejemplo en una inversión del control en manejador de eventos o máquinas de estados

14. Encadenamiento

- Cada método hace una tarea y encarga a otro método que haga el resto del proceso
- Se detecta fácilmente cuando la última sentencia de un método es la llamada a otro. El diagrama de módulos es muy vertical (una serie de cajas, cada una “colgada” de una superior).
- El diseño no es descendente, sino temporal, como “a trompicones”.

- Aunque puede que el programa se lea bien, los métodos carecen de cohesión, y no cumplen una tarea específica relacionada con el problema, sino con un período de tiempo del proceso.
- Los módulos serían difícilmente reutilizables.

15. El “main” falso

- El programa principal (“main”) delega todo su trabajo en otro método, de forma que consiste únicamente en una llamada a un método.
- Simplemente es innecesario.

16. Recursión innecesaria

- Utilizar recursión en un método cuando hay una solución iterativa natural.
- El peor caso es la recursión final, una mezcla de encadenamiento y recursión innecesaria: un método, como última acción, tiene una llamada a sí mismo.
- Los métodos recursivos son elegantes. Una vez que se aprende y entiende, la recursión es muy cómoda. Pero no debería emplearse como sustituto de la iteración.
- En general la recursión gasta más recursos (ocupación de la pila de forma indeterminada) y encubre una iteración.
- La recursión final es una clara iteración encubierta y siempre es evitable.
- Algunos problemas tienen fácil solución recursiva y muy difícil iterativa; estos son los mejores candidatos para emplear recursión.
- Ejemplo

```
procedure imprimirLineas (int n) {
    if (n>0) {
        System.out.println ();
        imprimirLineas (n-1);
    }
}
```

```
procedure imprimirLineas (int n) {
    for (int i = 1; i<=n; i++)
        System.out.println ();
}
```

17. “Abrir muchos teclados”

- Definir variables de tipo `Scanner` en distintos métodos.
- Por razones técnicas es preferible (en esta asignatura) pasarlo como parámetro si varios métodos van a tener entradas de usuario.